ICD-18

"The Gemini World Coordinate System"

Steven Beard
November 20, 1998

Version: E

Last Update: March 12, 2012

Issued By: System Group

Approved By: Date:

Group Manager

Manuel Lazo

Systems Engineer 2010/10/12

Gelys Trancho

Revision Control

1. Revision Version A
   Date: November 20 1998
   Revised by: Steven Beard
   Reason for / items changed: Initial version

2. Revision Version B
   Date: August 05 2010
   Revised by: Gelys Trancho
   Reason for / items changed: More complete version

3. Revision Version C
   Date: October 4 2010
   Revised by: Pedro Gigoux
   Reason for / items changed: Reorganized document. Added working example.

4. Revision Version D
   Date: October 12 2010
   Revised by: Gelys Trancho
   Reason for / items changed: Updated tables and add index of contents.

5. Revision Version E
   Date: March 20 2012
   Revised by: Pedro Gigoux
   Reason for / items changed: Fixed name of AO channel

1 Table of Contents

1 Table of Contents 3
2 Index of Tables 4
3 Description 5
4 Acronyms and Abbreviations 5
5 References 6
6 World Coordinate System 7
6.1 Inputs 7
6.2 Outputs 9
6.3 TCS Context 10
6.4 WCS Calibration File 11
7 Examples 13
7.1 WCS Code 13
7.2 Channel Access Routines 20

2 Index of Tables

Table 6.11: Telescope System Inputs 7
Table 6.12: Instrument Inputs 7
Table 6.21: WCS Output FITS Keywords 8
Table 6.22: RADECSYS Possible Values 8
Table 6.31: TCS Context 9

3 Description

This Interface Control Document (ICD) serves the following purposes:

- To describe the commands and parameters that necessary to construct the WCS for any Gemini Instrument
- To provide the necessary declarations of commands, parameters and Status and Information records

4 Acronyms and Abbreviations

EPICS	Experimental Physics and Industrial Control System
ICS	Instrument Control System
IOC	Input Output Controller
TCS	Telescope Control System
WCS	World Coordinate System
Channel Access	EPICS communication protocol
JCA	Java Channel Access
CAJ	Channel Access for Java
PHU	Primary Header Unit
ASTLIB	Astronomical Calculations Library
SLALIB	Positional Astronomy Library
TIMELIB	Time Library
TAI	International Atomic Time
TT	Terrestrial Time
UTC	Coordinated Universal Time
TDB	Barycentric Dynamical Time

5 References

(1) TN-PS-G0045, "Astronomical Coordinate Frames I: Astrometry" (tcs_ptw_008)
(2) TN-C-G0070, "The Flow of WCS Information through a Gemini ICS" (cics_smb_042)
(3) TN-PS-G0071, "Astronomical Coordinate Frames II: Quick Look Support" (tcs_ptw_009)

6 World Coordinate System

The science dataset must be calibrated to allow conversion of focal plane x, y positions to right ascension and declination. The conversion is dependent on the specifics of the instrument and telescope configuration at the time of the dataset's acquisition (although many of the parts of the conversion are constant.)

The production of WCS information is done in the instrument software by using values in a structure fetched from the TCS called the TCS Context, a WCS calibration file specific to the instrument, and other information also specific to the instrument. A detailed description of the TCS context and the WCS calibration file is given in Section 4.3 and 4.4, respectively

If an instrument needs more than one focal plane to sky transformation (for example for different wavelengths or chop states) then the WCS should be calculated separately for each case.

6.1 Inputs

There are two sources of inputs for the WCS calculation: telescope systems and instrument.

6.1.1 Telescope System Inputs

The telescope system inputs are all EPICS channels that should be retrieved using Channel Access. More details on how to do this can be found in the example.

EPICS Data Type	EPICS Channel Name	Description
STRING	tcs:sad:sourceAInputFrame.VAL	Track name. The following values are currently supported: ● AZEL_MNT ● AZEL_TOPO ● APPT ● FK5 ● FK4
STRING	tcs:sad:sourceAEpoch.VAL	Epoch. It's a string of the form "<char><equinox>", where <char> is a single character ('B' or 'J'), and <*equinox*> is a floating point number (for instance, 2000.0).
DOUBLE	tcs:sad:sourceAWavelength.VAL	Track wavelength in Angstroms
DOUBLE[45]	tcs:ak:astCtx.VALA	TCS context (array).
INTEGER	ag:port:<inst>	ISS port number.

EPICS Data Type	EPICS Channel Name	Description
	where <inst> is the instrument name assigned by Gemini in lowercase.	
STRING	tcs:drives:agAoName	AO fold position. If the value is **IN** then the AO system is in use.

Table 6.11: Telescope System Inputs

6.1.2 Instrument Inputs

Instrument inputs are stored inside the instrument. It's up to the instrument builder how to make them available to the WCS routine.

Data Type	Description
INTEGER	Binning in x
INTEGER	Binning in x
INTEGER	Start of ROI in x, in pixel coordinates
INTEGER	End of ROI in y, in pixel coordinates
INTEGER	Size of ROI in x, in pixels
INTEGER	Size of ROI in y, in pixels
INTEGER	Chop state where the WCS refers to: ● 0 = chop A ● 1 = chop B ● 2 = chop C Set it to zero (chop A) if not chopping.
STRING	WCS calibration file name (see section 6.4 for more details).

Table 6.12: Instrument Inputs

6.1.3 Time

In those cases where a time card is available, the time can be retrieved directly from the Gemini Time Bus. Otherwise, the time can be simulated. TAI is a sensible choice and is used in the example on Section 5. Whatever timescale is specified, it is important that it be continuous across a leap second. Suitable alternatives are TT and UT1.

TAI is used to calculate TDB-TT and the equation of the equinoxes in the TIMELIB routines, therefore it does not need to have a high accuracy.

It is important to note that the WCS routine should be called as close as possible to the start of the observation.

6.2 Outputs

All the outputs of the WCS calculation should be written to the image header. There will be a separate set of keywords per detector in the instrument. WCS keywords should be written to the header of the image extension corresponding to the detector and not the Primary Header Unit (PHU).

Keyword Name	Data Type	Value(s)	Comment
CTYPE1	STRING	AZ---TAN RA---TAN	WCS projection type, axis 1
CRPIX1	DOUBLE	---	Pixel coordinate reference, axis 1
CRVAL1	DOUBLE	---	World coordinate reference, axis 1
CRTYPE2	STRING	EL---TAN DEC--TAN	WCS projection type, axis 2
CRPIX1	DOUBLE	---	Pixel coordinate reference, axis 2
CRVAL2	DOUBLE	---	World coordinate reference, axis 2
CD1_1	DOUBLE	---	WCS matrix element 1 1
CD1_2	DOUBLE	---	WCS matrix element 1 2
CD2_1	DOUBLE	---	WCS matrix element 2 1
CD2_2	DOUBLE	---	WCS matrix element 2 2
RADECSYS	STRING	See table below	R.A./DEC. coordinate system reference

Table 6.21: WCS Output FITS Keywords

Track Name	RADECSYS
AZEL_MNT	Empty value
AZEL_TOPO	Empty value
APPT	GAPPT

FK5	FK5
FK4	FK4

Table 6.22: RADECSYS Possible Values

6.3 TCS Context

The TCS context is an array of 45 values. The following table contains the elements present in the TCS context as available directly from the TCS (tcs:ak:astCtx.VALA). This table is given as a reference only since the WCS routine does not need to deal with these elements directly.

0	ctx.time	Time stamp (TAI)
1-3	x, y, z	Cartesian elements of mount pre-flexure az/el
4	ctx.tel.fl	Telescope focal length (mm)
5	ctx.tel.rma	Rotator mechanical angle (rads)
6	ctx.tel.an	Azimuth axis tilt NS (rads)
7	ctx.tel.aw	Azimuth axis tilt EW (rads)
8	ctx.tel.pnpae	Az/el non-perp (rads)
9	ctx.tel.ca	Left-right collimation (rads)
10	ctx.tel.ce	Up-down collimation (rads)
11-24	ctx.aoprms	Target independent apparent-to-observed parameters
25-30	ctx.m2xy	M2 tip/tilts
31	ctx.po.mx	Mount pointing origin in X
32	ctx.po.my	Mount pointing origin in Y
33	ctx.po.ax	Source chop A pointing origin in X
34	ctx.po.ay	Source chop A pointing origin in Y
35	ctx.po.bx	Source chop B pointing origin in X
36	ctx.po.by	Source chop B pointing origin in Y
37	ctx.po.cx	Source chop C pointing origin in X
38	ctx.po.cy	Source chop C pointing origin in Y
39-44	ao2t	Optical distortion coefficients (not

		used to date)

Table 6.31: TCS Context

6.4 WCS Calibration File

The WCS calibration file is a four column text file containing the measured positions of a star on the chip in pixel coordinates (i,j) and the corresponding position of the star at the focal plane (x,y). The routines in ASTLIB are used to calculate the affine transformation between the two systems of coordinates.

The calibration file name should adhere to the following convention:

<ao><inst>_<port>_<detector|camera|etc..>.wcs

where:

<ao> is “ao” when the AO fold is in use, or empty otherwise
<inst> is the Gemini name of the instrument in lowercase
<port> is the port number where the instrument is installed on
<detector> is the detector number (can be omitted in the case of a a single detector)
<camera> is the camera type in case two o more are use in such instruments (can be omitted in the case of a single camera)

7 Examples

The examples included here were tested with the Gemini telescope systems. It is divided in two modules:

1. The WCS code that should be included in the instrument software
2. EPICS Channel Access routines written using the native libca library. Instrument builders are free to choose the interface that better suits them (e.g. JCA, CAJ, C++)

In order to compile and run this code the user needs to have the EPICS base, and the astlib, slalib and timelib libraries installed.

7.1 WCS Code

```
#include <stdio.h>
#include <string.h>
#include <astLib.h>
#include <slalib.h>
#include <timeLib.h>
#include "ca.h"

/*
 * Maximum number of elements
 */
#define NPOINTS             100         /* max # of points in a calibration file */
#define MATRIXSIZE          6           /* transformation matrix */
#define MAX_LINE  100       /* characters in an input line */

/*
 * Handy error codes
 */
#define    OK               0
#define ERROR               (-1)

/*
 * EPICS channel names. The strings "tcs:" and "ag:" are known as the
 * database tops. In this examples they are hardcoded in the channel names,
 * but (ideally) they should parameters that can be changed at run time
 * without having to recompile the code.
 */
#define TCS_TRACK_NAME  "tc1:sad:sourceAInputFrame.VAL"
#define TCS_EPOCH           "tc1:sad:sourceAEpoch.VAL"
#define TCS_WAVELENGTH  "tc1:sad:sourceAWavelength.VAL"
#define TCS_CONTEXT         "tc1:ak:astCtx.VALA"
#define AO_IN_USE           "tcs:drives:agAoName"
#define ISS_PORT  "ag:port:%s"          /* instrument name is a parameter */

/*
 * Instrument name. Change!
 */
#define INSTRUMENT          "test"

/*
 * Focal plane to detector transformation stuff
 */
double      fpxy[NPOINTS][2];   /* defined focal plane XY coords*/
double      pixij[NPOINTS][2];  /* measured (unbinned and unwindowed) pixel coords*/
double      cij[MATRIXSIZE];    /* XY to IJ transformation matrix. */
int         numPoints = 0;              /* number of points read */


/*
 * Read the instrument focal plane to detector surface calibration
```

```
 * measurements from a file.   This is done on initialization
 * or whenever a new set of calibration measurements have been made.
 *
 * According to Gemini ICD18 the calibration procedure is as follows:
 *
 * Command the TCS to put a star at position (x,y) in the focal plane,
 * (fpxy[p][0]), fpxy[p][1]).
 * Measure the position of that star on the detector in pixels (i,j),
 * (pixij[p][0], pixij[p][1]).
 *
 * Four points are specified in the gemini technical note tn_g_0045,
 * these are the mid-points betewen the center of the detector and
 * the four corners. It is assumed that the center pixel of the
 * detector has been aligned with the optical axis.
 *
 * The detector is read full-frame (no binning or windowing).
 *
 * Dummy numbers are use here, add file reading later...
 *
 */
static int readCalibration (const char *fname)
{
    int                 n, count;
    char    line[MAX_LINE];
    char    *p;
    FILE    *fp;

    if (fname == NULL) {
            printf ("readCalibration: null file name\n");
            return ERROR;
    }
    if (*fname == '\0') {
            printf ("readCalibration: empty file name\n");
            return ERROR;
    }

    numPoints = 0;

    if ((fp = fopen (fname, "r")) == NULL) {
            printf ("readCalibration: file does not exist (%s)\n", fname);
            return ERROR;
    }

    for (n = 0; n < NPOINTS; n++) {

            p = fgets (line, MAX_LINE, fp);
            if (p == NULL) {
               if (feof (fp)) {
                       break;
               } else {
                       /* ERROR error while reading file */
                       fclose (fp);
                       return ERROR;
               }
            }

            count =  sscanf(line, "%lf %lf %lf %lf",
                                    &(pixij[n][0]), &(pixij[n][1]),
                                    &(fpxy[n][0]), &(fpxy[n][1]));
            if (count != 4) {
               n--;
               continue;
            }
    }

    fclose (fp);
    numPoints = n;
```

```
    return OK;
}


/*
 * Function wcsCalibrate() defines a pixel coordinate (i,j) to (x,y)
 * transformation.
 *
 * It is assumed the calibration measurements were made with the
 * detector generating unbinned full frames. If the detector is reconfigured
 * to bin the data or generate a smaller region of interest, the (i,j)
 * values need to be transformed to match the detector coordinates in the
 * new frame of reference.
 *
 * For example, if the detector is skipping the first "iskip" pixels and
 * binning the remaining pixels by a factor "ibin" each i measurement becomes
 *
 * i_new = ((i_old - 0.5 - iskip) / ibin) + 0.5;
 */
long wcsCalibrate (void)
{
    int wcsStat;        /* WCS status */

    int p;              /* Point counter */

    int ibin, jbin;     /* Detector i and j binning factors */
    int iskip, jskip;   /* Pixel to skips in i and j (ROI) */

    double detij[NPOINTS][2]; /* Array of binned and windowed pixel (i,j) */

    double pixis;       /* x to i scale factor. */
    double pixjs;       /* y to j scale factor. */
    double perp;        /* Non-perpendicularity of i and j axes in radians. */
    double orient;      /* Orientation of (i,j) axes with respect to (x,y) (rad). */

    /*
     * If the detector is configured for binning and regions of interest
     * then these settings should be updated from instrument status variables
     * at this point. Use default values if the instrument has no binning
     * or roi capabilities.
     */
    ibin = 1;
    jbin = 1;
    iskip = 0;
    jskip = 0;

    /*
     * Transform the values to match the region of insters and the binning
     * settings in the instrument.
     */
    for (p= 0; p < numPoints; p++) {
            detij[p][0] = ((pixij[p][0] - 0.5 - (double) iskip) /
                                    (double) ibin) + 0.5;

            detij[p][1] = ((pixij[p][1] - 0.5 - (double) jskip) /
                                    (double) jbin) + 0.5;

    }

    /*
     * Use function astFitij() to generate an (i,j) to (x,y)
     * transformation.
     */
    wcsStat = astFitij (numPoints, fpxy, detij, cij, &pixis, &pixjs,
                                    &perp, &orient);
```

```
    if (wcsStat != 0) {
            /* ERROR failed to define (i,j) to (X,Y) transformation */
            return ERROR;
    }

    return OK;
}

/*
 * Get the current TCS context and the (i,j) to (x,y) fit
 * and calculates the WCS header items.
 *
 * This is the code that should be executed whenever the getWcs
 * command is received
 *
 */
long updateWcs (void)
{
    /* wcs calculation related stuff */
    struct WCS_CTX ctx;         /* World Coordinate System context. */
    struct WCS                  wcs;        /* Basic TCS World Coordinate System. */
    struct WCS                  wcsij;      /* WCS in detector frame of reference. */

    char    ctype1[81];         /* WCS projection type, axis 1. */
    double  crpix1;             /* WCS pixel coordinate reference, axis 1. */
    double  crval1;             /* WCS coordinate reference, axis 1. */
    char    ctype2[81];         /* WCS projection type, axis 2. */
    double  crpix2;             /* WCS pixel coordinate reference, axis 2. */
    double  crval2;             /* World coordinate reference, axis 2. */
    double  cd1_1;              /* xi rotation/skew matrix element. */
    double  cd1_2;              /* xj rotation/skew matrix element. */
    double  cd2_1;              /* yi rotation/skew matrix element. */
    double  cd2_2;              /* yj rotation/skew matrix element. */
    char    radecsys[81];       /* Type of RA/Dec (for celestial coord.) */

    /* time related stuff */
    double timeTAI;   /* International atomic time in secs. */
    double rawTimeWcs;          /* Gemini raw time at which WCS info valid*/

    /* telescope context related stuff */
    double              tcsCtx[AST_CTXA_SIZE];      /* Telescope context */
    int                         chopState;                  /* Chop state. */
    double              equinox;            /* Epoch of mean equator & equinox (celestial coords). */
    double              trackWavelength;    /* Track wavelength in microns. */
    FRAMETYPE                   trackFrame;                 /* TCS track frame. */
    struct EPOCH        trackEquinox;               /* TCS track equinox. */

    /* other stuff */
    double mjdobs;                                  /* Modified Julian Date. */
    int                 wcsStat;                    /* WCS status. */
    char    trackFrameString[41];                   /* track frame */
    char    trackEqCurrent[41];

    /*
     * Parameters for timeOffline. In this example TAI (MJD) is
     * set to a fixed value for simplicity.
     */
    double tai          = 55410.715336;     /* TAI (MJD) */
    double elong        = -70.736694;       /* east longitude Gemini South */
    double phi          =-30.24075;         /* lattitude of Gemini South */
    double hm           = 2722.0; /* height Gemini South */
    double dleap        = 999999.900;       /* MJD following lastest leap second */
    double dat          = 32.000; /* TAI-UTC before that date (sec) */
    double dut          = -0.353;  /* UT1-UTC before that date (sec) */

    /*
     * Simulate the Gemini TCS time service in order to be able to run
```

```
 * the time library functions offline.
 */
wcsStat = timeOffline (tai, elong, phi, hm, dleap, dat, dut);

/*
 * Read track frame (string) from the TCS
 */
if (getEpicsString (TCS_TRACK_NAME, trackFrameString) == ERROR) {
        printf ("updateWcs: error when reading track name\n");
        return ERROR;
}
if (strstr ("AZEL_MNT", trackFrameString) != NULL)
        trackFrame = AZEL_MNT;
else if (strstr ("AZEL_TOPO", trackFrameString) != NULL)
        trackFrame = AZEL_TOPO;
else if (strstr ("APPT", trackFrameString) != NULL)
        trackFrame = APPT;
else if (strstr ("FK5", trackFrameString) != NULL)
        trackFrame = FK5;
else if (strstr ("FK4", trackFrameString) != NULL)
        trackFrame = FK4;
else {
        printf ("updateWCS: invalid trackFrame (%s)\n", trackFrameString);
        return ERROR;
}
printf ("trackFrameString=(%s), trackFrame=%d\n",
            trackFrameString, trackFrame);

/*
 * Read epoch from the TCS. The epoch is a string like, for instance,
 * "J2000.0". The equinox would 2000 (float) and the type 'J' (char).
 */
if (getEpicsString (TCS_EPOCH, trackEqCurrent) == ERROR) {
        printf ("updateWCS: error while reading track name\n");
        return ERROR;
}
sscanf (&(trackEqCurrent[1]), "%lg", &equinox);
trackEquinox.year = equinox;
trackEquinox.type = trackEqCurrent[0];
printf ("type=%c, year=%lg\n", trackEquinox.type, trackEquinox.year);

/*
 * Read track wavelength (double) from the TCS (tcs:sourceAWavelength.VAL).
 * It's read as string and then converted to floatig point (double).
 */
if (getEpicsDouble (TCS_WAVELENGTH, &trackWavelength) == ERROR) {
        printf ("updateWCS: error when reading track wavelength\n");
        return ERROR;
}
printf ("trackWavelength=%g\n", trackWavelength);

/*
 * Read tCS context from the TCS.
 */
if (getEpicsDoubleArray (TCS_CONTEXT, tcsCtx, AST_CTXA_SIZE)
        == ERROR) {
        printf ("updateWCS: error when reading tcs context\n");
        return ERROR;
}

/*
 * Convert tracking wavelength from angstroms to microns
 */
trackWavelength = trackWavelength / 10000.0;
printf ("wavelength(um)=%g\n", trackWavelength);

/*
```

```
 * Set the chop state to which the WCS coordinate information refers.
 * This settings should be updated from instrument status variables
 * at this point. If no chopping is suported then set it to default
 * state A (0);
 */
chopState = 0;

/*
 * Set the context information needed for generating world coordinate
 * system transformations. The input for this routine should be the
 * context from the TCS (double array).
 */
(void) astSetctx (tcsCtx);
wcsStat = astGetctx (&ctx);
if (wcsStat != 0) {
        printf ("updateWcs: astGetctx failed to get current WCS context\n");
        return ERROR;
}

/*
 * Convert the WCS context into a focal plane WCS transform.
 * It returns the pointer to the WCS structure (struct WCS) and the
 * Gemini raw time at which the transformation was correct (double).
 */
wcsStat = astCtx2tr (ctx, trackFrame, trackEquinox, trackWavelength,
                                chopState, &wcs, &rawTimeWcs);
if (wcsStat != 0) {
        printf ("updateWcs: astCtx2tr failed to transform FP to sky\n");
        return ERROR;
}

/*
 * Convert the raw telescope time at which the WCS context was
 * received to TAI time (MJD).
 */
if (timeThenD (rawTimeWcs, TAI, &timeTAI) != 0) {
        printf ("updateWcs: timeThenD failed to convert time stamp to TAI\n");
        return ERROR;
}
printf ("timeTAI=%g\n", timeTAI);

/*
 * Update the instrument calibration (in case the readout parameters
 * have changed) then extend the FP WCS transform to the detector surface
 */
wcsStat = wcsCalibrate();
if (wcsStat != 0) {
        printf ("udateWcs: failed to update calibration\n");
        return ERROR;
}

/*
 * Extend a world coordinate system transformation to start from
 * instrumental i/j instead of focal-plane x/y. It is supposed to return
 * OK(0) all the time, but the check for error is done just in case.
 */
wcsStat = astXtndtr (cij, wcs, &wcsij);
if (wcsStat != 0) {
        printf ("updateWcs: astXtndtr failed to combine models\n");
        return ERROR;
}

/*
 * Express a world coordinate system transformation as FITS header strings.
 */
wcsStat = astFITSv (wcsij, trackFrame, trackEquinox, timeTAI,
                                ctype1, &crpix1, &crval1,
```

```
                                        ctype2, &crpix2, &crval2,
                                        &cd1_1, &cd1_2, &cd2_1, &cd2_2,
                                        radecsys, &equinox, &mjdobs);
    if (wcsStat != 0) {
            printf ("updateWcs: failed to calculate FITS header values\n");
            return ERROR;
    }

    /*
     * THE CODE TO UPDATE THE IMAGE HEADER INFORMATION SHOULD GO HERE
     */
    printf ("ctype1   = (%s)\n", ctype1);
    printf ("crpix1   = %g\n", crpix1);
    printf ("crval1   = %g\n", crval1);
    printf ("ctype2   = (%s)\n", ctype2);
    printf ("crpix2   = %g\n", crpix2);
    printf ("crval2   = %g\n", crval2);
    printf ("cd1_1    = %g\n", cd1_1);
    printf ("cd1_2    = %g\n", cd1_2);
    printf ("cd2_1    = %g\n", cd2_1);
    printf ("cd2_2    = %g\n", cd2_2);
    printf ("radecsys = (%s)\n", radecsys);

    return OK;
}

int main (int argc, char *argv[])
{
    int             detNum;
    long    port;
    char    auxstr[40];
    char    fname[200];
    char    ao[40];

    /*
     * Get the iss port where the instrument is installed
     */
    sprintf (auxstr, ISS_PORT, INSTRUMENT);
    if (getEpicsLong (auxstr, &port) == ERROR) {
            printf ("main: error when reading iss port\n");
            return ERROR;
    }
    printf ("port=%d\n", port);

    /*
     * Get detector number for which this routine is being called.
     * In this example we'll assume that detector number is 2
     */
    detNum = 2;

    /*
     * Check whether AO is being used.
     */
    if (getEpicsString (AO_IN_USE, ao) == ERROR) {
            printf ("main: error when reading ao setup\n");
            return ERROR;
    }
    printf ("port=%d\n", port);

    /*
     * Read wcs configuration file. The file name should be constructed
     * using the instrument name, detector number, and AO setting.
     */
    if (strstr (ao, "IN") != NULL)
            sprintf (fname, "ao%s_%d_%d.wcs", INSTRUMENT, port, detNum);
    else
            sprintf (fname, "%s_%d_%d.wcs", INSTRUMENT, port, detNum);
```

```
    if (readCalibration (fname) == ERROR) {
            printf ("Failed to read WCS file %s\n", fname);
            return ERROR;
    }

    updateWcs ();

    return OK;
}
```

7.2 Channel Access Routines

7.2.1 C Code (ca.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <cadef.h>

#define    OK                   0
#define ERROR                   (-1)

#define TIMEOUT                 5.0         /* seconds */
#define    PRIORITY             10          /* 0=lowest, 99=highest */

static int epicsGet (const char *, chtype, void *);

/* epicsContextCreate -- Initialize channel access context
 */
int epicsContextCreate ()
{
    if (ca_context_create (ca_disable_preemptive_callback) != ECA_NORMAL)
            return ERROR;
    else
            return OK;
}

/* epicsContextDestroy -- Destroy channel access context
 */
void epicsContextDestroy ()
{
    ca_context_destroy ();
}

/* getEpicsString -- Get string value
 */
int getEpicsString (const char *chan, char *value)
{
    chid     cid;

    *value = '\0';

    return epicsGet (chan, DBR_STRING, (void *) value);

    if (epicsSearch (chan, &cid) == ERROR)
            return ERROR;
    if (ca_get (DBR_STRING, cid, value) != ECA_NORMAL)
            return ERROR;
    if (ca_pend_io (TIMEOUT) != ECA_NORMAL)
            return ERROR;

    return OK;
}
```

```
/* getEpicsInt -- Get integer value
 */
int getEpicsInt (const char *chan, int *value)
{
    chid      cid;

    *value = 0;

    return epicsGet (chan, DBR_INT, (void *) value);
}

/* getEpicsLong -- Get long integer value
 */
int getEpicsLong (const char *chan, long *value)
{
    chid      cid;

    *value = 0L;

    return epicsGet (chan, DBR_LONG, (void *) value);
}

/* getEpicsDouble -- Get double value
 */
int getEpicsDouble (const char *chan, double *value)
{
    chid      cid;

    *value = 0.0;

    return epicsGet (chan, DBR_DOUBLE, (void *) value);
}

/* getEpicsDoubleArray -- Get double array
 */
int getEpicsDoubleArray (const char *chan, double *value, int count)
{
    chid      cid;

    if (value == NULL)
            return ERROR;

    memset (value, 0, count * sizeof (double)); /* clear buffer */

    if (ca_create_channel (chan, NULL, NULL, PRIORITY, &cid) != ECA_NORMAL)
            return ERROR;
    if (ca_pend_io (TIMEOUT) != ECA_NORMAL)
            return ERROR;
    if (ca_array_get (DBR_DOUBLE, count, cid, (void *) value) != ECA_NORMAL)
            return ERROR;
    if (ca_pend_io (TIMEOUT) != ECA_NORMAL)
            return ERROR;
    return OK;
}

/* epicsGet -- Get generic type channel (single value).
 */
static int epicsGet (const char *chan, chtype type, void *value)
{
    chid      cid;

    if (value == NULL)
            return ERROR;

    if (ca_create_channel (chan, NULL, NULL, PRIORITY, &cid) != ECA_NORMAL)
            return ERROR;
    if (ca_pend_io (TIMEOUT) != ECA_NORMAL)
```

```
        return ERROR;

    if (ca_get (type, cid, value) != ECA_NORMAL)
        return ERROR;
    if (ca_pend_io (TIMEOUT) != ECA_NORMAL)
        return ERROR;

    return OK;
}

#ifdef TESTING
/* Used for testing of this module
 */
int main (int argc, char *argv[])
{
    int         n;
    int         ival;
    long    lval;
    double  dval;
    char    sval[50];
    double  ctx[50];
    chid    cid;

    epicsContextCreate ();

    getEpicsInt ("ec:present", (void *) &ival);
    printf ("ival=%d\n", ival);

    getEpicsLong ("ec:thermalplcHeartbeat", (void *) &lval);
    printf ("lval=%d\n", lval);

    getEpicsDouble ("ec:domePos", (void *) &dval);
    printf ("dval=%g\n", dval);

    getEpicsString ("ec:domemotor.MESS", (void *) &sval);
    printf ("sval=(%s)\n", sval);

    getEpicsDoubleArray ("tcs:ak:astCtx.VALA", (void *) ctx, 45);
    for (n = 0; n < 45; n++)
        printf ("%d: %g\n", n, ctx[n]);

    epicsContextDestroy ();
}
#endif
```

7.2.2 Include File (ca.h)

```
#ifndef INCLUDE_CA_H
#define INCLUDE_CA_H

void epicsContextDestroy ();
int getEpicsString (const char *chan, char *value);
int getEpicsInt (const char *chan, int *value);
int getEpicsLong (const char *chan, long *value);
int getEpicsDouble (const char *chan, double *value);
int getEpicsDoubleArray (const char *chan, double *value, int count);

#endif
```